

Ejercicio del Curso de Representación de Conocimientos usando PROLOG como lenguaje y sistema de representación de conocimientos que implementa un subconjunto de la Lógica de Predicado de Primer Orden (LPP0)

Instrucciones

1. Descargue Prolog desde el sitio: www.swi-prolog.org
2. Instale el sistema en su carpeta de aplicaciones
3. Invoque SWI-Prolog haciendo click en el ícono de la aplicación, lo cual abrirá una ventana con el editor de SWI-Prolog con el prompt ?- en dicho editor, luego del cual se pueden ingresar expresiones (formas lógicas) para su evaluación en el sistema de representación de conocimientos.
4. Las expresiones lógicas se deben siempre ingresar en el editor luego del prompt ?- y deben ser finalizadas con un . y luego se deben ingresar con un retorno de carro para su evaluación en el sistema.

Preguntas

1. Por qué la siguiente expresión es verdadera?

?- $5 < 4$.

Su respuesta:

2. Qué significa la expresión siguiente y por qué es verdadera?

i. ?- $5 == 5$.

Su respuesta:

3. Por qué la siguiente expresión es verdadera?

?- $\text{juan} == \text{juan}$.

Su respuesta:

4. Por qué la siguiente expresión es falsa?

?- $5 == 4 + 1$.

Su respuesta:

5. Por qué la siguiente expresión es verdadera?

?- 5 =:= 4 + 1.

Su respuesta:

6. Ingrese la siguiente expresión y explique por qué ella es falsa:

?- 5 =\= 4 + 1.

Su respuesta:

7. Ingrese la siguiente expresión y explique por qué es falsa (el signo = debe ir en donde el signo < o > apunta):

?- 5 =< 4.

Su respuesta:

Nota: Ingresar expresiones directamente en el editor de SWI-Prolog es engorroso. Por ello, es preferible crear un archivo con las expresiones. Ese archivo corresponde a la “base de conocimientos” que usa el sistema de representación de conocimientos. Para crear dichos archivos puede usar cualquier editor de texto. La extensión del archivo debe ser .pl para indicar que es un archivo de Prolog.

En la ventana ingrese las siguientes expresiones en LPP0:

hombre(juan).
mujer(maria).
hijo(jose).

y luego almacene el archivo con un editor de texto y salvarlo con la extensión .pl y guardarlo en la misma ubicación, o sea, en: Users/< nombre de usuario>/Desktop/basedekonocimientos.pl

Acto seguido compile el archivo en el editor de SWI-Prolog con el siguiente comando, lo cual ingresa los datos en la bases de datos e instancia dichas reglas en el sistema de representación de conocimientos (el decodificador):

i. ?- consult('Users/< nombre de usuario>/Desktop/basedekonocimientos.pl').

Luego ingrese las siguientes expresiones en el editor de SWI-Prolog sin olvidar finalizar con . y retorno de carro:

- ii. hombre(juan).
- iii. mujer(maria).
- iv. hijo(jose).

Ahora ingresemos en lugar de constantes la variable X de la siguiente manera, con la X en mayúscula (la variable):

- v. ?- mujer(X).

Ahora ingrese un segundo hijo en la base de conocimientos (en el archivo basedekonocimientos.pl)

hijo(jorge).

y luego salve el archivo. Acto seguido compile el archivo en el editor de SWI-Prolog con el siguiente comando:

- vi. ?- consult('Users/< nombre de usuario>/Desktop/basedekonocimientos.pl').

Ahora ingrese la siguiente expresión en el editor de SWI-Prolog:

- vii. ?- hijo(X).

Como hay dos hijos en la base de conocimiento, el sistema entrega como respuesta el primer hijo en la base de conocimientos, es decir, jose.

Si ingresamos la letra *n*, es decir, *no*, en el editor de SWI-Prolog, el decodificador de Prolog entiende que no es esa la referencia anafórica correcta y nos entrega como resultado la segunda opción, es decir *jorge*.

Volvamos a ingresar la misma expresión:

- viii. ?- hijo(X).

Nuevamente, el decodificador de Prolog nos entrega la primera opción, es decir, *jose*, como resultado. Si ahora nosotros confirmamos con *y*, es decir, *yes*, en el editor de SWI-Prolog, el sistema confirma la opción de resolver la referencia anafórica con jose, es decir, se instancia la siguiente regla en el sistema de representación de conocimientos:

X = jose.

Qué pasa si Ud. Ahora ingresa la siguiente expresión en el editor de SWI-Prolog:

- ix. ?- hombre(jaime).

Prolog responde con el valor *false* porque no encuentra esa regla de conocimiento en la base de conocimientos.

8. Pregunta: Qué regla de razonamiento no monotónico está implementando Prolog para dar esa respuesta?

Ahora agreguemos la siguiente regla en la base de conocimientos:

pareja(juan,maria).

Almacene esta regla en la base de conocimientos y luego instancie (compile) esa regla en el sistema de representación de conocimientos usando el comando:

x. ?- consult('Users/< nombre de usuario>/Desktop/basedekonocimientos.pl').

Ahora ingrese la siguiente expresión en el editor de SWI-Prolog:

xi. ?- pareja(juan,maria).

Ahora ingrese la siguiente expresión en el editor de SWI-Prolog:

xii. ?- pareja(maria,juan).

Por qué esa expresion es falsa?

Para evitar el orden de los operandos podemos usar una regla que es almacenada en la base de conocimientos:

pareja(X,Y) :- pareja(Y,X).

Almacene esta regla en la base de conocimientos y luego instancie (compile) esta regla en el sistema de representación de conocimientos usando el comando:

xiii. ?- consult('Users/< nombre de usuario>/Desktop/basedekonocimientos.pl').

Ahora ingrese la siguiente expresión en el editor de SWI-Prolog:

xiv.?- pareja(maria,juan).

Ahora el sistema de representación de conocimientos nos entrega la respuesta "true" como se espera.

Prolog también se pueden ingresar expresiones con conectores lógicos. Por ejemplo, la siguiente expresión es un ejemplo del conector "and", lo que se representa en Prolog con el símbolo ",". Por ejemplo, la siguiente expresión es falsa:

xv. ?- true,false.

La siguiente expresión es un ejemplo del conector “or”, lo que se representa en Prolog con el símbolo “;”. Por ejemplo, la siguiente expresión es verdadera:

xvi.?- true; false.

Las listas representan un tipo de data abstracto que se representa en Prolog a través de la siguiente sintaxis:

[cabeza de la lista | resto de la lista]. Sin embargo la lista se expresa sin el caracter “|”.

Ejemplo:

La lista [1, 2, 3] representa la lista [1 | [2 |[3]]], en la cual a su vez la lista [2 |[3]] representa la lista [2, 3] (es decir, la estructura de datos “lista” es recursiva).

El primer elemento de la lista siempre representa la cabeza de la lista.

Por ejemplo, usando la función member en Prolog podemos escribir la siguiente expresión:

xvii. ?- member(3, [1, 2, 3]).

Por qué lo siguiente es falso?

xviii. ?- member(4, [1, 2, 3]).

Expresiones Complejas de Unificación

En Prolog el algoritmo de unificación es el más usado para razonamiento lógico y se denota por el símbolo “lado 1 = lado 2”. Este algoritmo se basa en dos reglas básicas.

- i. Si ambos lados son átomos (constantes) se unifican si son iguales
- ii. Si uno de los lados es una variable y el otro no, entonces la variable recibe ese valor del otro lado
- iii. Si ambos lados son variables entonces ambas variables apuntan al mismo registro de memoria por lo tanto cualquier valor que una asuma le asigna o cambia el valor a la otra
- iv. Si ambos lados son expresiones complejas entonces se aplican dos reglas:
 - a. Ambas expresiones deben asociarse al mismo predicado
 - b. Ambas expresiones deben tener la misma aridad

Veamos unos ejemplos:

Ingrese las siguientes reglas en la base de conocimientos:

```
% auto(patente, consumo, tipo de combustible, lista de choferes)
% chofer(nombre, años de experiencia, kilómetros conducidos)
```

```
auto(pat123, 0.4, bencina,
[chofer(antonio,10,10000),chofer(andres,20,20000),chofer(alfredo, 30, 30000)]).
auto(pat124, 0.3, diesel, [chofer(pedro,5,5000), chofer(pablo, 15, 15000)]).
```

Almacene como siempre estas reglas usando el editor de texto. Luego instancie (compile) estas reglas en el sistema de representación de conocimientos usando el siguiente comando en el editor SWI-Prolog:

```
xix.?- consult('Users/< nombre de usuario>/Desktop/basedekonocimientos.pl').
```

Ingrese en la consola del editor SWI-Prolog una regla para resolver la siguiente pregunta:

Cuáles son las patentes de todos los autos que consumen bencina? Considere que los argumentos que no interesan son anónimos, lo que se indica con el carácter “_”

Solución:

```
xx. ?- auto(Patente, _ bencina, _).
```

Escriba una regla en la base de conocimientos que entregue la cantidad de choferes ha que ha conducido el auto que tiene la patente que el usuario entre por consola.

La solución se puede obtener a través de la siguiente regla en la base de conocimientos:

```
cantidad :- write('Entre la patente del auto: '),
             read(Patente),
             auto(Patente, _ , ListaDeChoferes),
             length(ListaDeChoferes, Cantidad),
             write('La cantidad es: '),
             write(Cantidad).
```

Almacene como siempre esta regla en la base de conocimientos usando el editor de texto. Luego instancie (compile) estas reglas en el sistema de representación de conocimientos usando el comando en el editor SWI-Prolog:

```
xxi.      ?- consult('Users/< nombre de usuario>/Desktop/basedekonocimientos.pl').
```

Acto seguido en la consola del editor SWI-Prolog invoque dicha regla así:

```
xxii.     ?- cantidad.
```

Ingresando la patente seguida de “.” en la consola cuando el sistema se lo pida.

Escriba una regla en la base de conocimientos que dada la patente de un auto y una cantidad de bencina que le quede al auto calcule la cantidad de kilómetros que el auto puede todavía recorrer.

Solución:

```
cantidaddekilometros(Patente,Combustible,CantidadDeKilometros) :-  
auto(Patente, Consumo, _ _), CantidadDeKilometros is Consumo*Combustible.
```

Nota: El conector “is” le asigna a la variable CantidadDeKilometros el valor del resultado de la multiplicación de los valores de las variables Consumo y Combustible (pruebe con “=” a ver qué resultado le entrega el sistema)

Almacene como siempre esta regla usando el editor texto. Luego instancie (compile) estas reglas en el sistema de representación de conocimientos usando el comando en el editor SWI-Prolog:

```
xxiii. ?- consult('Users/< nombre de usuario>/Desktop/basedekonocimientos.pl').
```

Finalmente, en la consola invoque la regla de arriba así:

```
xxiv. ?- cantidaddekilometros(pat123, 20, CantidadDeKilometros).
```

La regla anterior devuelve el valor de la variable CantidadDeKilometros.